



Building and Running a Spring Boot Application Locally – Creative Theory Notes

1. Creating a Spring Boot Project

To build a Spring Boot application, the speaker uses **Spring Initializr** (start.spring.io), a web-based project generator.

Selected configurations:

- **Project:** Maven
- **Language:** Java
- **Spring Boot Version:** 3.1.5
- **Group ID:** `com.isco`
- **Artifact ID:** `rest-demo`
- **Packaging:** `Jar`

Packaging as a **jar file** is important because jar files are easy to run and ideal for deployment inside Docker containers.

2. Opening the Project in IDE

After generating the project:

- The ZIP file is extracted
- The project is opened in **IntelliJ IDEA Community Edition**

At this stage:

- The application is a basic Spring Boot setup
- No Docker integration is present yet
- Running the main class shows no visible output in the browser

This confirms the project skeleton is ready but lacks business logic.

3. Creating a REST Controller

To produce visible output, a REST endpoint is added.

A new class called **HelloController** is created:

- Annotated with `@RestController`
- Maps an HTTP request to the root URL (`/`)
- Returns a simple **"Hello World"** string

The `@RestController` annotation ensures:

- The response is sent directly to the client
 - Data is returned in JSON/text format without a view layer
-

4. Running the Application and Port Conflict

When the application is run locally:

- A **port conflict error** occurs
- The default Spring Boot port (8080) is already in use

Possible reasons:

- Another application is running
 - A Docker container is occupying the port
-

5. Resolving Port Conflict

To fix the issue, the speaker:

1. Stops conflicting containers
2. Changes the Spring Boot port in `application.properties`

The server port is updated to:

- **8081**

After restarting the application:

- The app runs successfully
- Accessing `http://localhost:8081` returns **"Hello World"**

This confirms the REST API is working correctly.

6. Preparing the Application for Packaging

Next, the focus shifts to **packaging the application**.

Key steps:

- Configure the final jar name as `rest-demo.jar`
- Optionally specify the main application class
- Refresh Maven dependencies

This ensures Maven generates a clean, executable jar file.

7. Building the Application with Maven

The application is packaged using Maven:

- Command used: **mvn package**
- Can be executed via:
 - IntelliJ Maven tool window
 - Terminal

The build completes successfully with no errors, generating the jar file.

8. Location of the Executable Jar

After packaging:

- The jar file is created inside the **target** folder
- File name: **rest-demo.jar**

This jar contains:

- Application code
- Embedded web server
- All required dependencies

It is fully executable.

9. Running the Jar File Locally

To test the packaged application, the speaker runs:

- **java -jar rest-demo.jar**

Results:

- Spring Boot application starts successfully
- Runs on default port **8080**
- Accessing the browser returns **"Hello World"**

This confirms the application can run independently without an IDE.

10. Why This Step Is Important

Running the jar file proves that:

- The application is production-ready
- No IDE dependency exists
- The app is suitable for containerization

This step is critical before deploying the app inside Docker.

11. Preparing for Docker Deployment

With:

- Docker already installed
- A running container environment
- The executable jar file ready

The next step is to:

- Copy the jar file into a Docker container
- Run the Spring Boot app inside the container

This bridges **local development** and **containerized deployment**.

12. Conclusion

Building and running a Spring Boot application locally ensures that the application logic, configuration, and packaging are correct. By generating an executable jar and verifying it outside the IDE, the application becomes ready for Docker-based deployment.

This section forms the foundation for **Dockerizing a Spring Boot application**.

If you want, next I can:

- Write the **Dockerfile for this Spring Boot app**
- Explain **COPY vs ADD in Docker**
- Show **port mapping (8080 → container)**
- Walk through **running Spring Boot inside Docker**
- Convert this into **exam notes or lab steps** 🐳☕